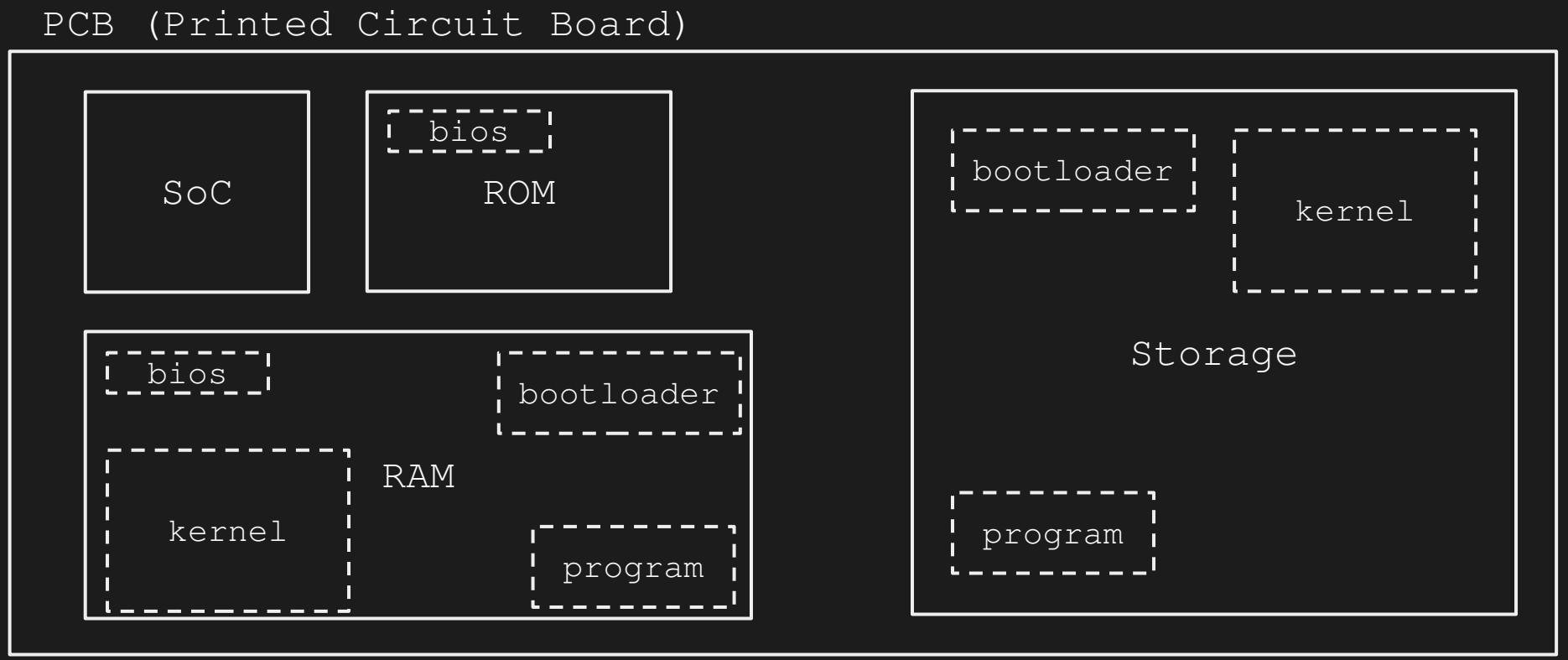
CPU Architecture

Hardware Workflow



Boot Stages

```
unsigned char x = 255;
short         y = 0x1337;
int           z = 0x00D34D00;
unsigned      w = 0x12345678;

char s[] = "0123";
```

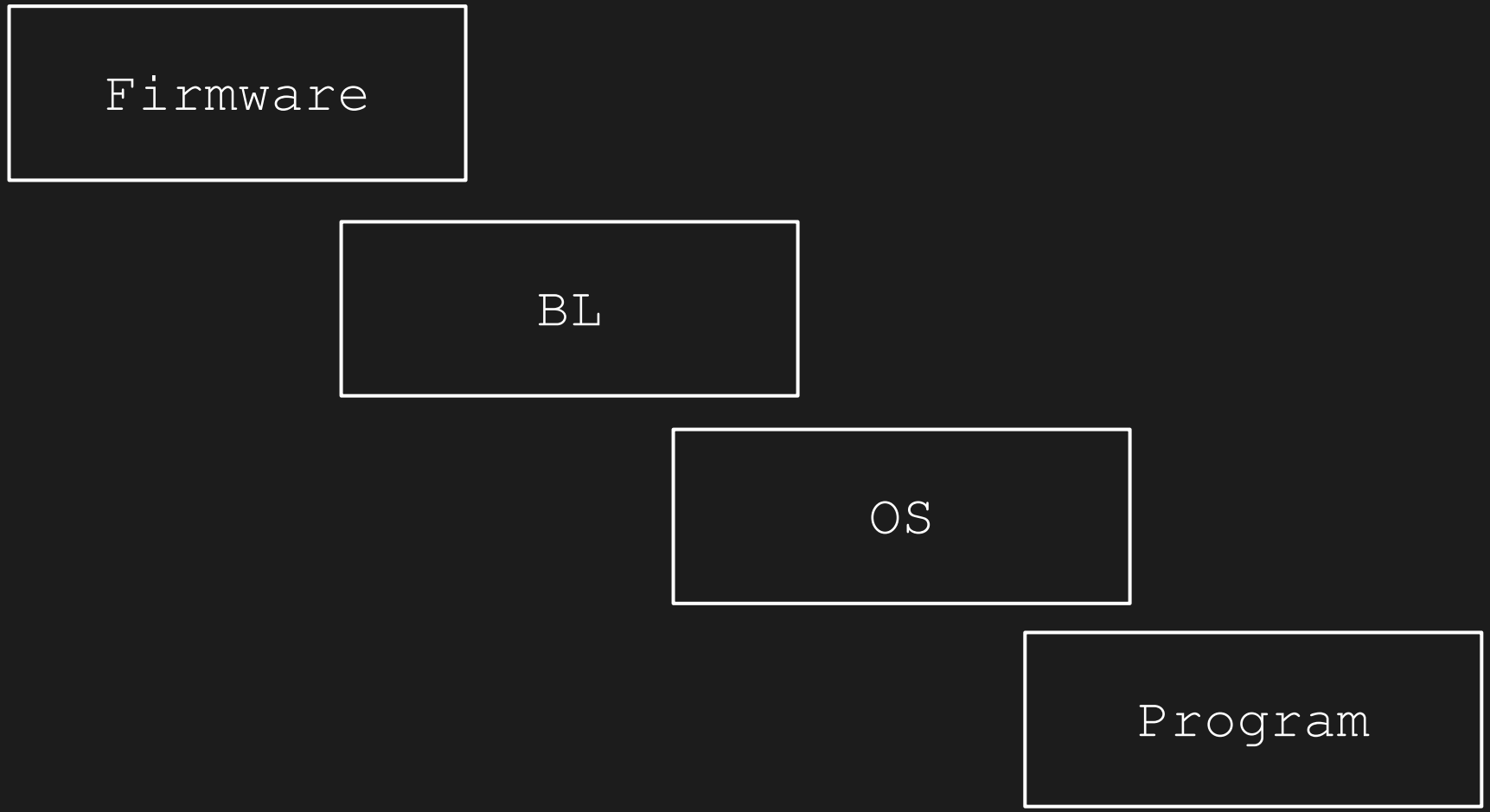
dump (BE)

```
Firmware
```

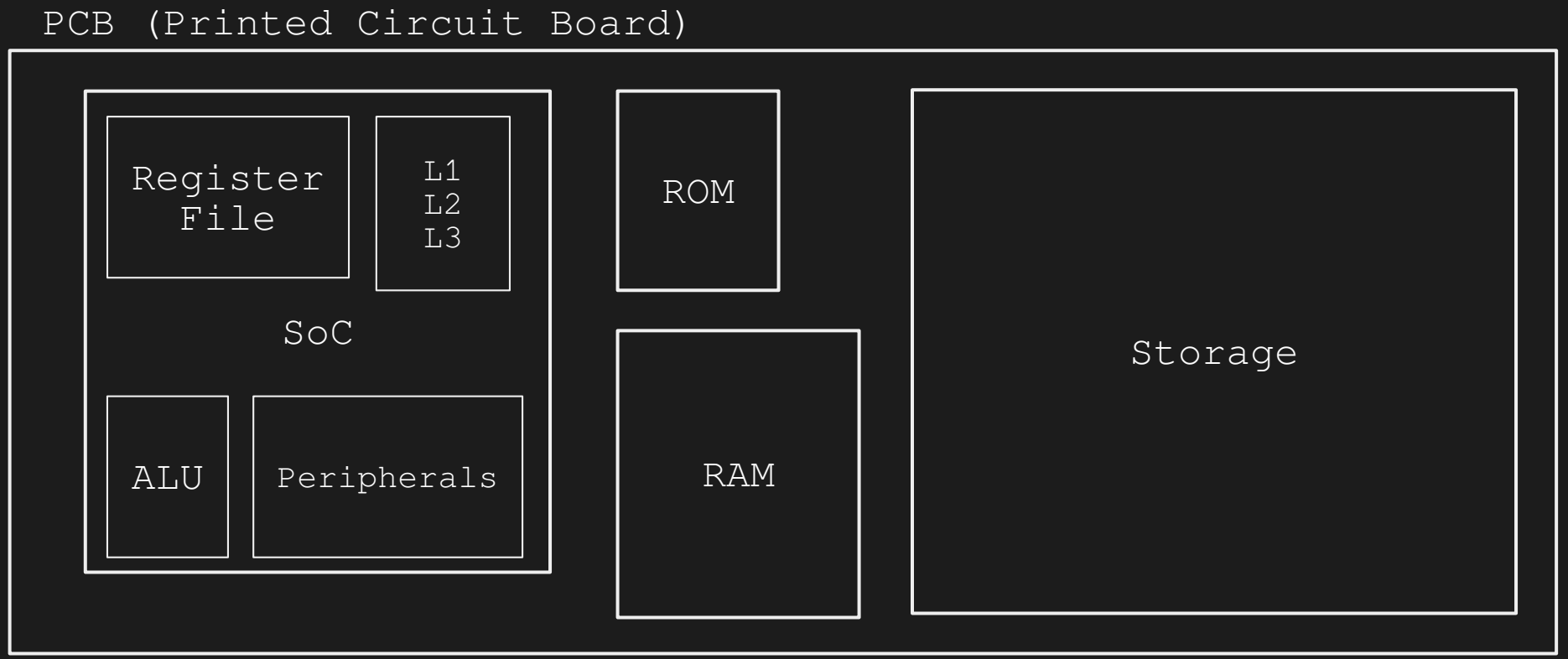
dump (LE)

```
... FF ...
... 37 13 ...
... 00 4D D3 00 ...
... 78 56 34 12 ...
... 30 31 32 33 ...
```

Boot Stages



Hardware Workflow



Architectures

[*] x86 → Intel, AMD
[*] ARM → STM, Apple, Qualcomm, ...
[*] RISC-V → SiFive, Microchip, Espressif, ...
[*] AVR → Atmel (Microchip)
[*] MIPS → Broadcom, Microchip, ...
[*] PIC → Microchip
[*] 68K → Motorola (NXP)
[*] PowerPC → IBM, NXP, Renesas, ...
[*] SPARC → Sun (Oracle), Fujitsu
[*] RX → Renesas
[*] Extensa → Cadence, Espressif, ...
[*] Itanium → Intel
[*] ...

Registers

[*] often basic memory units
[*] sometimes used for special operations
[*] used by the processing unit to perform tasks
[*] usually local variables

```
x86      →    rax, rbx, rcx, ...
arm      →    w0, w1, w2, ...
mips     →    v0, v1, t0, ...
risc-v   →    ra, t0, t1, ...
...
```

Code Instructions

```
amd64:
mov rax, 0          →       48 C7 C0 00 00 00 00
add rax, rbx        →       48 01 D8
```

```
aarch64:
mov x0, 0           →       00 00 80 D2
add x0, x0, x1      →       00 00 01 8B
```

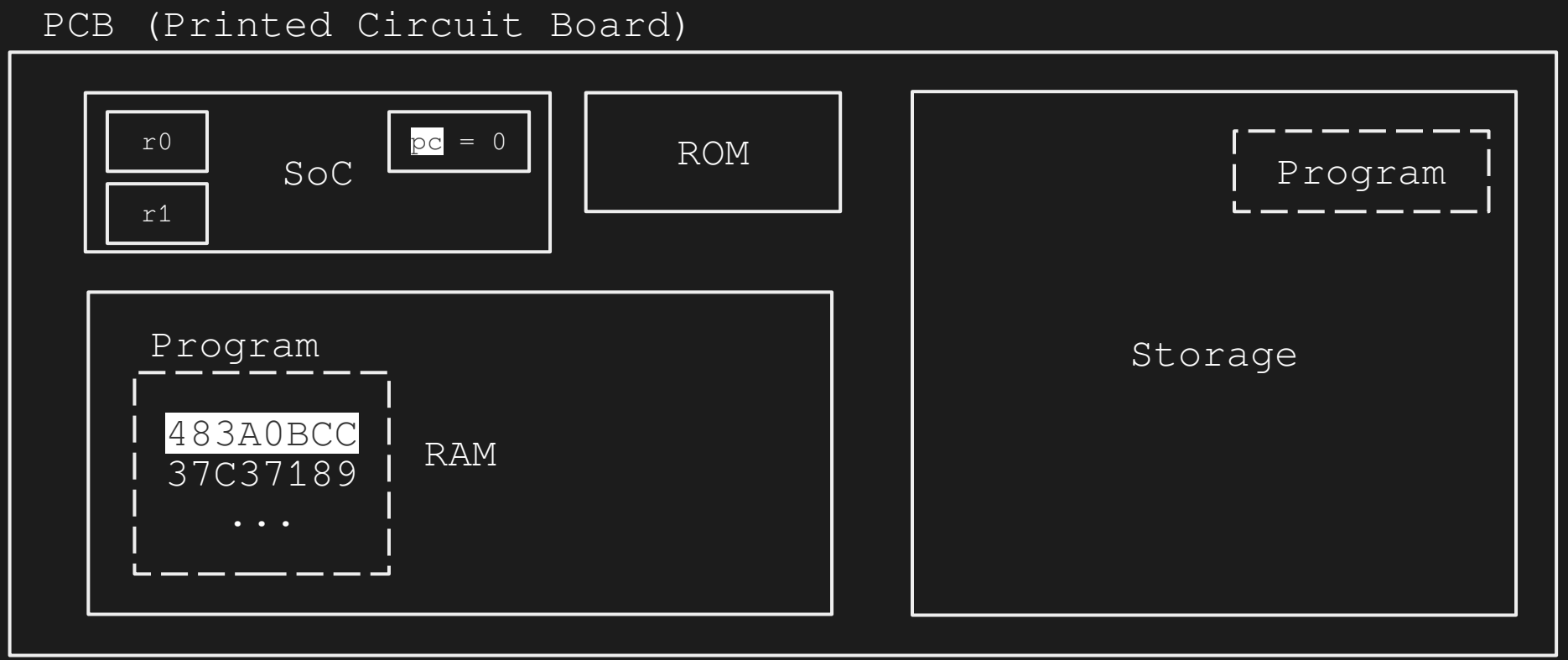
Instruction Mapping

```
my-arch-32:                          |op|rx|im|ry|
-------------------------------------------------
mov r0, 0                  →          00 00 00 00
mov r1, 0                  →          00 01 00 00
mov r2, 0                  →          00 02 00 00
mov r3, 0                  →          00 03 00 00
mov r0, 1                  →          00 00 01 00
add r0, 1                  →          01 00 01 00
add r1, r2                 →          01 01 00 02
sub r4, r3                 →          02 04 00 03
```

Basic Operations

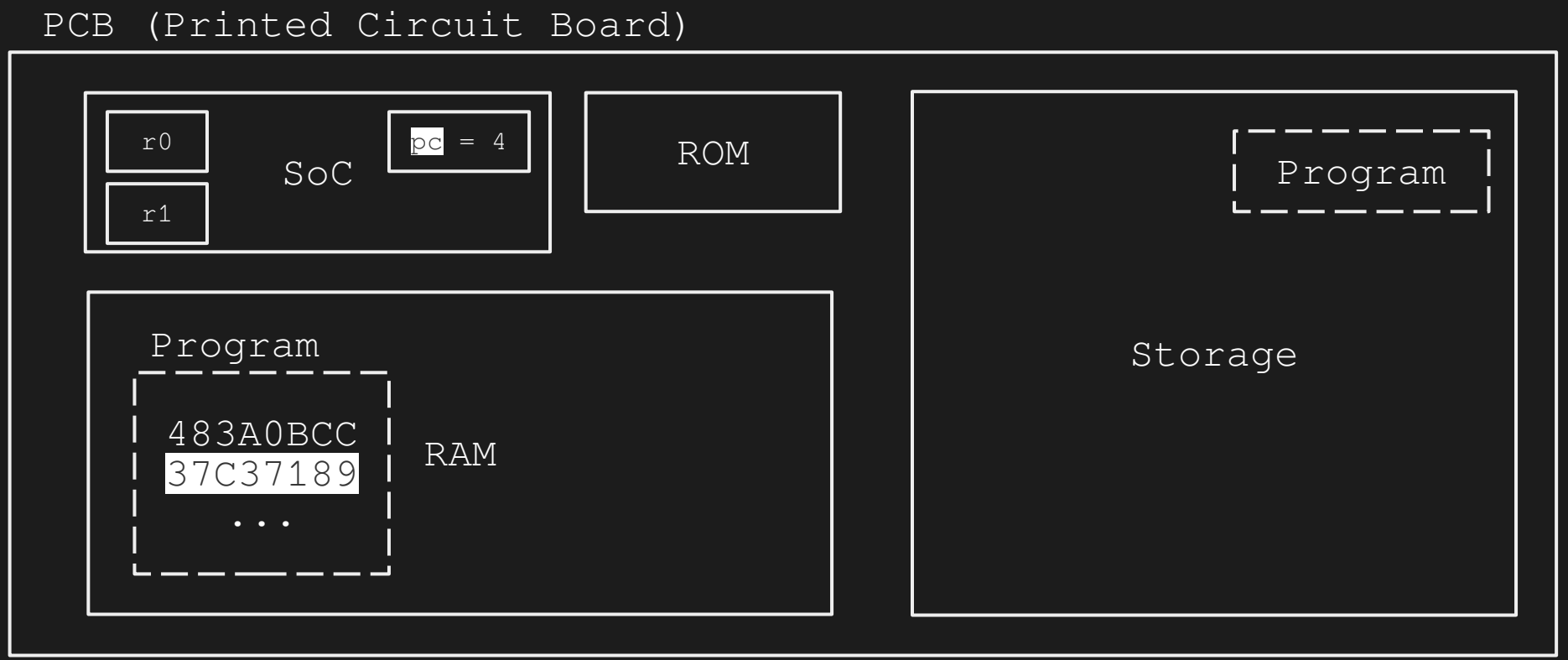
	x86	arm	mips	risc
x = a + b	**add** rax, rbx	**add** x0, x1, x2	**dadd** $v0, $a0, $a1	**add** a0, a1, a2
x = a + 2	**add** rax, 2	**add** x0, x1, #2	**daddi** $v0, $a0, 2	**addi** a0, a1, 2
x = a - b	**sub** rax, rbx	**sub** x0, x1, x2	**dsub** $v0, $a0, $a1	**sub** a0, a1, a2
x = a - 2	**sub** rax, 2	**sub** x0, x1, #2	**dsubi** $v0, $a0, 2	**addi** a0, a1, -2
x = a * b	**imul** rax, rbx	**mul** x0, x1, x2	**dmult*** $a0, $a1	**mul** a0, a1, a2
x = a / b	**div** rbx	**sdiv** x0, x1, x2	**ddiv*** $a0, $a1	**div** a0, a1, a2
x = a & b	**and** rax, rbx	**and** x0, x1, x2	**and** $v0, $a0, $a1	**and** a0, a1, a2
x = a | b	**or** rax, rbx	**orr** x0, x1, x2	**or** $v0, $a0, $a1	**or** a0, a1, a2
x = a ^ b	**xor** rax, rbx	**eor** x0, x1, x2	**xor** $v0, $a0, $a1	**xor** a0, a1, a2
x = a << 4	**shl** rax, 4	**lsl** x0, x1, #4	**dsll** $v0, $a0, 4	**slli** a0, a1, 4
x = a >> 4	**shr** rax, 4	**lsr** x0, x1, #4	**dsrl** $v0, $a0, 4	**srli** a0, a1, 4
x = a	**mov** rax, rbx	**mov** x0, x1	**or** $v0, $zero, $a0	**mv** a0, a1
x = 3	**mov** rax, 3	**mov** x0, #3	**daddiu** $v0, $zero, 3	**li** a0, 3
x = *a	**mov** rax, [rbx]	**ldr** x0, [x1]	**ld** $v0, 0($a0)	**ld** a0, 0(a1)
*x = a	**mov** [rax], rbx	**str** x1, [x0]	**sd** $a1, 0($a0)	**sd** a1, 0(a0)
x = a > b ?	**cmp** rax, rbx	**subs** xzr, x0, x1	**slt** $v0, $a0, $a1	**sub** a0, a1, a2
goto _LBL	**jmp** _LBL	**b** _LBL	**j** _LBL	**j** _LBL
goto _LBL ?	**je** _LBL	**b.eq** _LBL	**beq** $a0, $a1, _LBL	**beq** a0, a1, _LBL
function()	**call** function	**bl** function	**jal** function	**jal** ra, function
return	**ret**	**ret**	**jr** $ra	**ret**

Program Counter



* x86 → rip mips, arm, risc → pc

Program Counter



* x86 → rip mips, arm, risc → pc

Branches

amd64:

```
; Compares registers, updating RFLAGS (sets Zero Flag (ZF) if equal)
cmp rax, rbx
je  label
```

arm64:

```
; Branch if Equal, evaluates the Zero (Z) flag from NZCV
cmp  x1, x2
b.eq label
```

mips64:

```
; Branch if Equal, compares two registers directly
beq $t0, $t1, label
```

risc64:

```
; Branch if Equal: compares registers, branches if equal
beq x5, x6, label
```

Questions?